

Puesta en producción segura

Tarea 1: Prueba de aplicaciones web y para dispositivos móviles

Índice

Índice.....	1
Caso práctico.....	2
Apartado 1: Script.....	3
Crea un script que devuelva la cadena más larga de una lista de cadenas.....	3
Apartado 2: Testing.....	5
Crea un programa principal donde definiremos una clase llamada Test donde probaremos nuestro software,.....	5
Apartado 3: Verificación.....	7
Ejecuta el programa final y verifica si realmente el programa mychar.py se comporta como esperamos.....	7
Si el programa que comprueba el código detecta un error, ¿nos reflejará qué dato está esperando y que ha recibido?.....	8
¿Qué tipo de prueba hemos realizado?.....	9

Caso práctico

Julián cuando necesita comprobar parte de un código en Python usa la librería incluida por defecto en las distribuciones de Python llamada `unittest`.

Tiene un pequeño programa que solicita por teclado 5 palabras y devuelve la más larga. Para ello crea una pequeña función que recibe una lista de cadenas y devuelve la que tiene mayor longitud.

Para comprobar que todo funciona correctamente, decide hacer un test del software, es decir quiere verificar que una parte del código se comporta de forma esperada, para ello crea un script que importará la librería `unittest` y comprobará si la función que ha desarrollado como mínimo funciona correctamente para la lista `["a", "ab", "abc", "dddd", "abcd"]`.

Apartado 1: Script

Crea un script que devuelva la cadena más larga de una lista de cadenas.

Para realizar esta tarea es obligatorio que uses el lenguaje Python 3, por su facilidad y por el gran acceso a librerías de evaluación de software que posee. Si aún no has programado en Python, su inmersión te será sencilla y amigable.

- Llamaremos a este script `mychar.py`.
- Deberías de crear un programa con su estructura completa
- Dentro de este programa crearemos una función llamada `cadena_mas_larga` que reciba una lista de cadenas y devuelva la cadena más larga de la lista. Si dos o más cadenas tienen la misma longitud máxima, la función debe devolver la primera de ellas si las ordenamos alfabéticamente desde la A a la Z. Si la lista está vacía, la función debe devolver una cadena vacía (`""`).
- En el mismo fichero `mychar.py`, crea un programa que solicite al usuario 5 palabras y devuelva la más larga usando la función anterior.

```
def cadena_mas_larga(lista_cadenas):
    if not lista_cadenas:
        return ""

    # Encontrar la longitud máxima
    longitud_max = max(len(cadena) for cadena in lista_cadenas)

    # Filtrar las cadenas que tienen la longitud máxima
    cadenas_maximas = [
        cadena
        for cadena in lista_cadenas
        if len(cadena) == longitud_max
    ]

    # Devolver la primera cadena en orden alfabético
    return sorted(cadenas_maximas)[0]

def main():
    palabras = []
    for _ in range(5):
        palabra = input("Introduce una palabra: ")
        palabras.append(palabra)
```

```
    resultado = cadena_mas_larga(palabras)
    print("La cadena más larga es:", resultado)

if __name__ == "__main__":
    main()
```

Apartado 2: Testing

Crea un programa principal donde definiremos una clase llamada Test donde probaremos nuestro software,

- Importaremos la librería de texto de software. En este caso, `unittest`.
- Crearemos nuestra clase (del tipo `unittest.TestCase`).
- Dentro de esta clase, definiremos, al menos, una función para reflejar el tipo de testeo que vamos a realizar. En este caso, tu objetivo es verificar que, para la lista `["a", "ab", "abc", "dddd", "abcd"]`, la función devuelva `"abcd"`.
- No olvides incluir todas aquellas comprobaciones que creas necesarias para testear que la función `cadena_mas_larga` funciona correctamente. No des por sentado que el programador puede hacer un "buen uso" de ella. Prepárala para "lo peor".

```
import unittest
from mychar import cadena_mas_larga

class Test(unittest.TestCase):
    def test_cadena_mas_larga(self):
        # Caso básico
        self.assertEqual(cadena_mas_larga(["a", "ab", "abc", "dddd",
"abcd"]), "abcd")

        # Caso con cadenas de la misma longitud
        self.assertEqual(cadena_mas_larga(["abc", "def", "ghi"]), "abc")

        # Caso con lista vacía
        self.assertEqual(cadena_mas_larga([]), "")

        # Caso con una sola cadena
        self.assertEqual(cadena_mas_larga(["solo"]), "solo")

        # Caso con múltiples cadenas de diferentes longitudes
        self.assertEqual(cadena_mas_larga(["cort", "largo", "muylargo",
"extralargo"]), "extralargo")

        # Caso con cadenas que incluyen espacios
        self.assertEqual(cadena_mas_larga(["a b", "ab c", "abc d"]), "abc
d")

        # Caso con cadenas numéricas
        self.assertEqual(cadena_mas_larga(["123", "1234", "12"]), "1234")
```

```
        # Caso con caracteres especiales
        self.assertEqual(cadena_mas_larga(["!@#", "$%^&*", "()*+"]),
"$%^&*")

if __name__ == "__main__":
    unittest.main()
```

Apartado 3: Verificación

Ejecuta el programa final y verifica si realmente el programa `mychar.py` se comporta como esperamos.

Procedemos a ejecutar el script para comprobar que funciona correctamente:



López Henestrosa, José
Carlos

```
jose.lopez@SDGESP-MRW5C9JC tarea % python3 mychar.py
Introduce una palabra: largo
Introduce una palabra: extralargo
Introduce una palabra: muylargo
Introduce una palabra: cort
Introduce una palabra: cor
La cadena más larga es: extralargo
jose.lopez@SDGESP-MRW5C9JC tarea %
```

Como podemos apreciar, la cadena más larga es, efectivamente, **extralargo**.

A continuación, ejecutamos los tests para comprobar si hay algún caso que falle:



López Henestrosa, José
Carlos

```
jose.lopez@SDGESP-MRW5C9JC tarea % python3 -u test.py
.
-----
Ran 1 test in 0.000s

OK
jose.lopez@SDGESP-MRW5C9JC tarea %
```

De la misma forma, los tests funcionan correctamente, lo cual es el comportamiento esperado.

Si el programa que comprueba el código detecta un error, ¿nos reflejará qué dato está esperando y que ha recibido?

Vamos a modificar el archivo provisto en el apartado 2 para comprobar si se cumple lo que plantea el enunciado. En este caso, vamos a modificar el siguiente caso:

```
# Caso con una sola cadena
self.assertEqual(cadena_mas_larga(["a", "ab", "abc", "dddd", "abcd"]), "abcd")
```

Por este otro, el cual es **erróneo**:

```
self.assertEqual(cadena_mas_larga(["a", "ab", "abc", "dddd", "abcd"]), "dddd")
```

Ejecutamos el programa con el cambio realizado:



López Henestrosa, José Carlos

[✎ Editar perfil](#)

```
jose.lopez@SDGESP-MRW5C9JC tarea % python3 -u test.py
F
=====
FAIL: test_cadena_mas_larga (__main__.Test)
=====
Traceback (most recent call last):
  File "/Users/jose.lopez/Projects/Educacion/curso-especializacion-ciberseguridad-ti-p
rivado/puesta_en_produccion_segura/u1/tarea/test.py", line 7, in test_cadena_mas_larga
    self.assertEqual(cadena_mas_larga(["a", "ab", "abc", "dddd", "abcd"]), "dddd")
AssertionError: 'abcd' != 'dddd'
- abcd
+ dddd
=====
Ran 1 test in 0.000s
FAILED (failures=1)
```

Como podemos apreciar, el programa de testing nos detecta que hay un error en los tests. Refleja el dato que está esperando (Expected: 'abcd') y el dato que ha recibido (Received: 'dddd').

¿Qué tipo de prueba hemos realizado?

Se trata de un **test unitario**, el cual es una pequeña pieza de código diseñada para probar de manera independiente una función o método específico de una aplicación (en este caso, `cadena_mas_larga`).

Los tests unitarios ayudan a verificar que cada componente del código funcione correctamente y que los cambios futuros no introduzcan errores.